

Universidad de Carabobo
Facultad de Ciencias y Tecnología (FACYT)

Departamento de Computación
Asignatura: Arquitectura del Computador
4to Semestre

PRACTICA DE LABORATORIO 1

**Estudio Comparativo de Paridad:
Implementaciones Iterativas y Recursivas en
MIPS32**

Profesor:
José Canache

Estudiante:
Gabriel Vargas
C.I.: 32.035.567

Valencia, Junio de 2026

1. Introducción y Objetivos

El análisis del rendimiento algorítmico a bajo nivel constituye una de las bases fundamentales en el diseño de sistemas y la arquitectura de computadores. Los lenguajes de alto nivel ocultan la mecánica operativa del hardware, abstrayendo elementos críticos como la gestión de memoria y el direccionamiento físico. Esta práctica aborda la evaluación de dos paradigmas de control (iterativo y recursivo) mediante el conjunto de instrucciones MIPS32, utilizando el entorno de simulación MARS.

El propósito central es determinar la paridad de un número entero positivo n bajo la especificación formal provista por la cátedra. A través de este problema elemental, se evalúan conceptos avanzados como el manejo de subrutinas, las convenciones de resguardo de registros y la administración manual del espacio de memoria dinámica en el segmento de pila del procesador.

1.1. Objetivos Específicos

- Familiarizarse con la sintaxis, directivas de ensamblador y el repertorio de instrucciones de la arquitectura MIPS32.
- Comprender e implementar el mecanismo de subrutinas a través de instrucciones de salto y enlace (`jal`) y salto por registro (`jr`).
- Diseñar y gestionar manualmente el bloque de activación (*Stack Frame*) a través del registro de puntero de pila (`$sp`).
- Contrastar cuantitativa y cualitativamente las diferencias de consumo de recursos (ciclos de ejecución, accesos a memoria y uso de registros) entre un enfoque puramente iterativo y uno recursivo.

2. Respuestas al Cuestionario del Informe

2.1. Implementación de la recursividad en MIPS32 y el rol de la pila (\$sp)

Dado que la arquitectura RISC de MIPS32 no posee instrucciones de alto nivel dedicadas al manejo nativo de funciones recursivas, esta estructura de control debe implementarse de forma explícita por el programador combinando saltos incondicionales con la manipulación de la pila del sistema.

La instrucción clave es `jal` (*Jump and Link*). Al invocarse esta instrucción, el procesador realiza dos acciones de hardware de forma atómica: carga la dirección de la subrutina en el Contador de Programa (PC) y almacena automáticamente la dirección de retorno ($PC + 4$) en el registro `$ra` (*Return Address*). Cuando una función se llama a sí misma de manera recursiva, cada nueva ejecución de `jal` sobrescribe el valor previo de `$ra`, perdiendo irremediablemente el camino de regreso.

Para evitar la pérdida del flujo de retorno hacia las funciones precedentes, se emplea el registro `$sp` (*Stack Pointer*). El puntero de pila delimita un área de memoria dinámica (*Stack Segment*) que crece hacia direcciones descendentes. Cada nivel de llamada activa un marco de activación (*Stack Frame*) donde se resguardan los estados locales de ese nivel específico a través de instrucciones `sw` (*Store Word*), almacenando tanto la dirección de retorno `$ra` como el argumento actual de trabajo `$a0`. Al alcanzarse el caso base, el programa desapila los datos en orden inverso mediante `lw` (*Load Word*) y restaura el espacio sumando el desplazamiento al puntero, permitiendo que la instrucción `jr $ra` devuelva el flujo ordenadamente a la instancia superior.

2.2. Riesgos de desbordamiento de pila (*Stack Overflow*) y estrategias de mitigación

El riesgo crítico inherente a la recursividad en entornos de bajo nivel es el desbordamiento de la pila o *Stack Overflow*. En la arquitectura MIPS32, la pila posee límites definidos por el mapa de memoria del sistema. Cada nivel de recursión consume memoria de manera estrictamente lineal. Para un marco de activación estándar de 8 bytes (necesario para resguardar dos palabras de 32 bits), una entrada n demandará $8 \times n$ bytes en la memoria RAM.

Si el valor de entrada n es excesivamente grande, o si ocurre una falla lógica que impida alcanzar la condición del caso base, la pila continuará expandiéndose indefinidamente hacia direcciones bajas de memoria hasta colisionar con el segmento de datos dinámicos (*Heap*) o sobrepasar los límites de asignación del sistema operativo del simulador, resultando en una excepción por direccionamiento inválido y el aborto de la aplicación.

2.2.1. Mitigación del riesgo

Para mitigar este fallo, se deben aplicar las siguientes directrices de diseño a nivel de software y hardware:

1. **Validación Rigurosa de Entrada:** Implementar filtros condicionales al inicio del programa principal (*main*) para verificar que el argumento cumpla con las precondiciones matemáticas, validando que no sea negativo y estableciendo un techo máximo tolerable para el argumento de entrada.
2. **Garantía de Convergencia del Caso Base:** Asegurar que todas las bifurcaciones lógicas decrezcan de forma estricta hacia la condición de parada. En este ejercicio, decrementar uniformemente el argumento asegura alcanzar la condición cero que detiene la recursión.

2.3. Diferencias operativas en el uso de memoria y registros

La discrepancia entre los enfoques iterativo y recursivo en MIPS32 representa el balance clásico entre la eficiencia de hardware y la abstracción algorítmica.

2.3.1. Gestión de Memoria RAM

La solución iterativa exhibe una eficiencia óptima de espacio, operando con una complejidad espacial de $O(1)$. No requiere modificar el registro `$sp` ni interactuar con la memoria volátil durante el procesamiento del bucle, puesto que todo el control ocurre internamente en los registros del procesador. Por el contrario, la solución recursiva opera con una complejidad espacial de $O(n)$, donde cada paso intermedio incurre en retardos físicos de acceso a memoria debido al tráfico constante generado por las instrucciones de lectura y escritura en la pila (`sw` y `lw`).

2.3.2. Gestión de Registros

En la solución iterativa, los registros mantienen sus estados de manera continua a lo largo de las iteraciones del lazo, lo que minimiza drásticamente los accesos al bus de datos. En la solución recursiva, la convención de MIPS obliga a tratar los registros de argumentos y valores temporales como volátiles a través de las llamadas. Por lo tanto, el procesador se ve forzado a realizar un vaciado (*register spilling*) hacia la memoria RAM para salvaguardar la coherencia de los datos entre llamadas anidadas.

2.4. Diferencias entre ejemplos académicos del libro de texto y un programa operativo en MARS

Los textos teóricos de referencia, tales como el de David A. Patterson y John L. Hennessy (*Estructura y diseño de computadores: la interfaz hardware/software*), poseen un enfoque netamente didáctico y pedagógico. Su prioridad es aislar el repertorio

Cuadro 1: Cuadro Comparativo de Recursos: Iterativo vs. Recursivo en MIPS32

Criterio de Evaluación	Implementación Iterativa	Implementación Recursiva
Complejidad Espacial	$O(1)$ – Constante. No consume espacio adicional.	$O(n)$ – Lineal. Crece proporcionalmente al argumento n .
Uso del Puntero \$sp	Nulo. El registro \$sp permanece inalterado.	Intensivo. Se modifica dos veces por cada llamada recursiva.
Tráfico en el Bus de Datos	Mínimo. Limitado a la carga inicial de variables.	Crítico. Multiplicidad de operaciones <code>sw</code> y <code>lw</code> en la RAM.
Ciclos de Reloj por Etapa	Muy bajo. Saltos directos cortos mediante <code>j</code> .	Alto. Penalizaciones por llamadas <code>jal</code> , <code>jr</code> y accesos a memoria.
Ciclo de Vida de Registros	Persistente dentro del contexto local del ciclo.	Volátil. Obliga al resguardo continuo en el <i>Stack Frame</i> .

de instrucciones para explicar la lógica pura del procesamiento aritmético o de control, omitiendo los componentes infraestructurales requeridos para la ejecución real en un computador. Al contrastar los fragmentos provistos en la literatura teórica con los códigos operativos reales desarrollados para MARS, se identifican divergencias estructurales críticas.

La mayor diferencia radica en la finalización limpia. Un ejemplo de libro simplemente asume que el código deja de ejecutarse al terminar la función. En un entorno real como MARS, si se omitiera el bloque de finalización por llamada al sistema (servicio 10), el procesador continuaría decodificando de forma secuencial las posiciones de memoria subsiguientes, interpretando datos estáticos o instrucciones vacías (`nop`) como código de operación válido, desencadenando fallos catastróficos de ejecución.

2.5. Tutorial de ejecución paso a paso en el entorno MARS

Para validar y analizar los algoritmos desarrollados de forma externa, el entorno de desarrollo MARS proporciona herramientas de depuración a bajo nivel. A continuación, se estructura el protocolo sistemático para la ejecución controlada de los programas:

1. **Carga y Ensamblado del Archivo:** Abrir el entorno MARS, cargar el archivo con extensión `.asm` y ejecutar el proceso de ensamblado presionando el icono del martillo y la llave inglesa (o la tecla de acceso rápido **F3**). Este paso convierte el código fuente en lenguaje máquina y cambia la interfaz a la pestaña *Execute*.
2. **Reconocimiento del Segmento de Texto y Datos:** En la vista central se desplegará el *Text Segment*, donde se visualizan cuatro columnas esenciales: la dirección física de memoria (en formato hexadecimal, ej. `0x00400000`), el código de

Cuadro 2: Divergencias Estructurales: Modelos de Libros vs. Código Operativo

Componente	Ejemplos de Libros (Teoría)	Código Fuente Operativo (Práctica)
Segmentación	Omitida. Se listan secuencias directas de instrucciones aisladas.	Obligatoria. División en bloques jerárquicos <code>.data</code> y <code>.text</code> .
Gestión de Datos	Abstracta. Se asume que los valores ya residen en los registros.	Explícita. Declaración de directivas como <code>.word</code> y <code>.ascii</code> en memoria estática.
Punto de Entrada	Indefinido. La ejecución inicia implícitamente en la primera línea expuesta.	Formalizado. Uso estricto de la etiqueta global <code>main</code> y la declaración de inicio.
Finalización	Inexistente o mediante retornos vacíos de subrutina.	Explícita. Llamadas al sistema operativo mediante servicios específicos (<code>li \$v0, 10; syscall</code>).
Interactividad	Ignorada por completo para evitar ruido visual en las explicaciones.	Presente. Empleo de llamadas al sistema (<code>syscall</code>) para la salida de texto en consola.

máquina correspondiente en hexadecimal, la instrucción en lenguaje ensamblador limpio y la línea de código fuente original escrita por el programador. En la sección inferior, la pestaña *Data Segment* permite monitorear los cambios de las variables almacenadas en la memoria estática (como la constante `n`).

3. **Ejecución Controlada Paso a Paso (*Single-Stepping*):** En lugar de ejecutar todo el programa de manera continua con el botón de flecha verde, se debe presionar el icono de la flecha con el número uno o pulsar la tecla **F7** (*Step*). Cada pulsación ejecuta una única instrucción de hardware, permitiendo observar la sincronía entre el flujo de control y las variaciones internas del procesador.
4. **Monitoreo en Tiempo Real de Registros y la Pila:** Durante el paso a paso del archivo recursivo, se debe prestar atención al panel lateral derecho (*Registers*). Al ejecutar instrucciones de reserva de espacio, se observará cómo el registro `$sp` disminuye su valor desde su dirección base inicial (`0x7ffffffc`), reflejando el crecimiento físico de la pila. Asimismo, el registro `$v0` cambiará dinámicamente entre 0 y 1 a medida que los retornos evalúan la expresión aritmética de alternancia de paridad.

2.6. Justificación de la elección del enfoque según criterios de eficiencia y claridad

Para la resolución del problema de determinación de paridad en la arquitectura MIPS32, la elección óptima e inapelable desde la perspectiva de la ingeniería de hardware es el ****enfoque iterativo****.

2.6.1. Criterio de Eficiencia

La implementación iterativa optimiza drásticamente el uso de los componentes físicos del computador. Al confinarse en el banco de registros, elimina por completo las penalizaciones por latencia de memoria asociadas al acceso de la memoria caché y la memoria RAM principal. Además, reduce el conteo total de instrucciones ejecutadas por el procesador, evitando la sobrecarga estructural impuesta por la creación de marcos de activación, saltos remotos y operaciones de restauración de contexto de registros. El ahorro en ciclos de reloj y en ancho de banda del bus de datos es masivo en comparación con la recursividad cuando n escala.

2.6.2. Criterio de Claridad

Si bien a nivel lógico y matemático la recursividad ofrece una traducción casi literal de la fórmula dada, en el lenguaje ensamblador MIPS32 esta elegancia se desvanece debido a la complejidad de la gestión manual de la pila. El enfoque iterativo estructurado mediante un ciclo condicional limpio (**for**) resulta significativamente más legible, directo, fácil de documentar y menos propenso a errores humanos críticos como la omisión accidental del resguardo del registro de retorno **\$ra**.

2.7. Análisis y Discusión de los Resultados

Las pruebas experimentales computadas dentro del simulador MARS arrojaron resultados idénticos para ambos algoritmos bajo valores controlados de entrada, validando la consistencia matemática de ambas soluciones. Para un valor de prueba de $n = 2$, ambos programas imprimieron con éxito en la consola el valor 0, confirmando correctamente que el número evaluado es de paridad par.

Sin embargo, el análisis del comportamiento interno reveló diferencias fundamentales durante el paso a paso:

- El programa iterativo completó el cálculo recorriendo el bucle exactamente dos veces mediante saltos locales (**j**), manteniendo constante el uso de los recursos de hardware y sin alterar la memoria externa.
- El programa recursivo generó un total de tres marcos de activación en la pila antes de poder resolver el problema. Se observó cómo el puntero de pila **\$sp** decreció sucesivamente pasando por los valores **0x7fffffff8**, **0x7ffffff0** y **0x7fffffe8**

para almacenar los contextos de $n = 2$, $n = 1$ y el caso base $n = 0$, respectivamente. Solo tras alcanzar el caso base, la paridad comenzó a propagarse de vuelta hacia las instancias superiores alternando el resultado en `$v0`, evidenciando de forma empírica el elevado costo que la recursividad impone sobre el subsistema de memoria.

El desarrollo práctico permitió verificar que la optimización a bajo nivel exige priorizar arquitecturas algorítmicas que minimicen el tráfico en el bus de datos. Se concluye que el paradigma iterativo representa la solución más balanceada y de mayor rendimiento para sistemas de cómputo con recursos restringidos o de propósito general gobernados por el conjunto de instrucciones MIPS32.